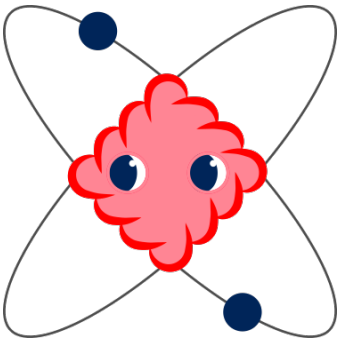


berry suite of programs

Documentation

v.2.0

July 2, 2026

Contents

Introduction

This guide gives a general introduction to the **berry** suite of programs, including instalation and running. An introduction to the basics of electronic structure calculations is given, in the context of this suite. Another introduction (for version 1.0) can be found in

```
Berry: A code for the differentiation of Bloch wavefunctions
       from DFT calculations
Computer Physics Communications 295 (2024) 10897
```

1 What berry does

The **berry** suite of programs extracts the Bloch wavefunctions from DFT calculations in an ordered way so they can be directly used to make calculations.

In practice it retrieves the wavefunctions and their gradients in reciprocal space totally ordered by unentangled bands, where continuity (analyticity) applies.

With the Bloch wavefunctions, berry can calculate:

- Berry connections
- Berry curvatures
- Linear optical conductivity
- Second harmonic generation (SHG) optical conductivity

It can be used in one, two or three dimension materials, with noncolinear or no-spin calculations.

For the DFT calculations, Quantum Espresso[5, 6] has to be used, as of version 2 of the program.

2 People

The **berry** suite is coordinated by Ricardo Mendes Ribeiro (University of Minho, Braga, Portugal), which is also the main developer.

Other contributors include

- Irving Leander Reascos Valencia (Braga, Portugal) for improving the performance and AI work;
- Daniel Sousa (Braga, Portugal) for implementing the noncolinear code and extension to 1D and 3D materials;
- Fábio Carneiro (Braga, Portugal) for the parallelization of the python code and its performance;
- Nuno Castro (LIP-Minho, Braga, Portugal) for coordinating AI work;
- Gonçalo Ventura (Porto, Portugal) for the equations for non-linear optical properties calculated from the Berry connections;
- Ícaro Jael Moura (Fortaleza, Brasil) for the exploratory work and testing;
- Afonso Duarte Ribeiro, author of the logo.

3 Contacts

The web site for the **berry** suite is

https://ricardoribeiro-2020.github.io/berry/

All files for instalation can be found there.

There is no maillling list yet for reporting bugs or other questions, but an email can be sent to ricardo.ribeiro@physics.org or you can create an issue in the github page.

4 Aknowledgement

We aknowledge the Fundação para a Ciência e a Tecnologia (FCT) under project QUEST2D *Excitations in quantum 2D materials* PTDC/FIS-MAC/2045/2021
and in the framework of the Strategic Funding UIDB/04650/2020.

Instalation and running

1 PIP install

To install the **berry** suite a pip install command is enough:

```
pip install berry-suite
```

Alternatively, it can be downloaded from the github
(https://ricardoribeiro-2020.github.io/berry/)
extracted to a directory and from that directory run the command:

```
pip install -e .
```

For developers, the developer dependencies can be installed using:

```
pip install berry-suite[dev]
```

or

```
pip install -e .[dev]
```

1.1 Patch for noncolinear calculations

As for the time of this version, the Quantum ESPRESSO suite of programs does not include a script to correctly extract the spinors of the noncolinear calculation in real space.

The script that should do it is `wfck2r.x`, but it only writes the first function of the spinor. In order to use the **berry** correctly in noncolinear calculations, a modified version of `wfck2r.x` has to be used.

In appendix A, we explain how to do it.

2 Requirements

This version of **berry** only supports the DFT package Quantum Espresso, version v.6.6 or higher. So a working Quantum Espresso instalation has to be in the system, and must be in the $PATH.

This version was tested with python 3.10, but likely works with other versions of python3.

3 Running

To run, first one has to create a working directory where the results will be saved, and inside it create a directory called *dft*.

Inside *dft* should go the pseudopotential files for the atomic species of the dft calculation and the file *scf.in* with the details of the scf run. You can use another name for the file, but then you have to add a line in the input file of the script `preprocess` to change the default (see section Preprocessing).

This *scf.in* file has to be a QUANTUM ESPRESSO scf run file; this is the only one implemented so far.

Create an input file (as described in chapter Workflow, section Preprocessing) in the working directory.

Scheme of working directory:

```
working_directory|
                 |input_file
                 |dft|
                     |scf.in
                     |pseudopotentials.upf
```

All the scripts should be run from the working directory. To run, a command of the form:

```
berry [package options] script parameter [script options]
```

where the `script` is one of the following

```
preprocess
wfcgen
degenrot
dot
cluster
basis
r2k
geometry
condutivity
shg
```

and the parameter is dependent on which script is beeing run. The command `berry` is a command line interface (CLI) of the package.

These scripts should be run in sequence, in the order shown, and the `berry` command assures that all the needed files are present for each run. The package options are shown in table 2.1.

Chapter Workflow explains what each script does, what parameters are required and what options they have.

The flowchart of the previous page explains how **berry** works.

First a preprocessing script is run which runs the DFT calculations, reads data from those calculations, and saves it in files ready to be read by the folllowing scripts.

Then another script reads the wavefunctions from the DFT calculations and make them coherent. This is done with the help of a script from the DFT suite.

The main part is the one that classifies each wavefunction into a band, assuring continuity. Once this classification is done, the gradient in reciprocal space of the Bloch factors of the wavefunctions can be calculated.

After that, Berry connections and curvatures, as well as other calculations can be performed.

berry flow chart

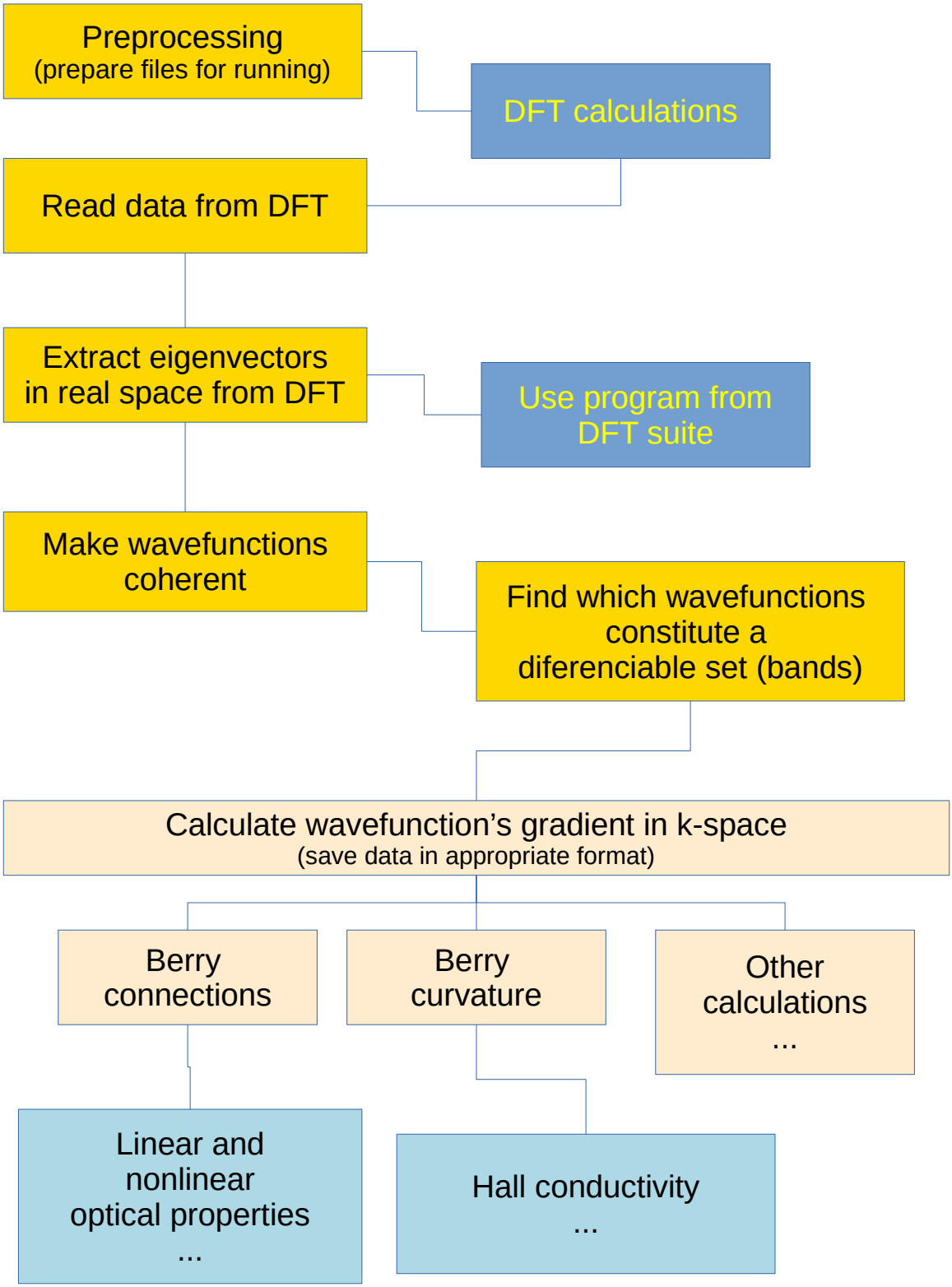


Table 2.1: The berry package options.

`-h`, `--help`	Shows this help message and exit
`--version`	Displays current Berry version

Theoretical background to *berry*

1 Electronic structure calculations basics

Crystal

A crystal is a set of atoms, forming a pattern, that is replicated all over space in one, two or three dimensions. It forms a crystal *lattice*.

Here we will deal only with 2D materials, so the pattern, which is called *unit cell*, is repeated in two dimensions, in a plane.

The unit cell is defined by three vectors (two in 2D) $\boldsymbol{a}_1$, $\boldsymbol{a}_2$ and $\boldsymbol{a}_3$, in position space (the real space). One can go from one unit cell to any other one by aplying the translation

$$\boldsymbol{R} = n_1\boldsymbol{a}_1 + n_2\boldsymbol{a}_2 + n_3\boldsymbol{a}_3 \tag{3.1}$$

where n_1, n_2 and n_3 are integers, and $\boldsymbol{R}$ is called the lattice vector.

A Fourier transform of this periodic pattern gives another periodic pattern in the so-called *reciprocal space* or momentum space. This new periodicity is defined by the reciprocal vectors:

$$\boldsymbol{b}_1 = 2\pi\frac{\boldsymbol{a}_2 \times \boldsymbol{a}_3}{\boldsymbol{a}_1 \cdot (\boldsymbol{a}_2 \times \boldsymbol{a}_3)}; \qquad \boldsymbol{b}_2 = 2\pi\frac{\boldsymbol{a}_3 \times \boldsymbol{a}_1}{\boldsymbol{a}_1 \cdot (\boldsymbol{a}_2 \times \boldsymbol{a}_3)}; \qquad \boldsymbol{b}_3 = 2\pi\frac{\boldsymbol{a}_1 \times \boldsymbol{a}_2}{\boldsymbol{a}_1 \cdot (\boldsymbol{a}_2 \times \boldsymbol{a}_3)}$$

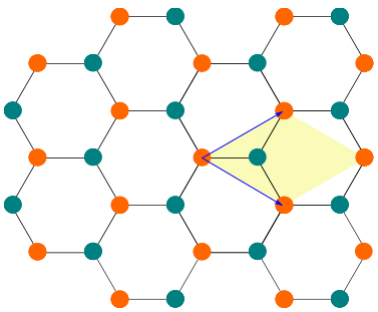


Figure 3.1: Example of a unit cell in a two dimensional material, with hexagonal symmetry. The yellow shadow represents the unit cell and the two blue vectors represent the lattice vectors.

Electrons in a crystal

Electrons in a crystal can be described by the solutions to time independent Schrödinger equation:

$$H_{\boldsymbol{k}}\psi_{\boldsymbol{k},s}(\boldsymbol{r}) = E_{\boldsymbol{k},s}\psi_{\boldsymbol{k},s}(\boldsymbol{r}) \tag{3.2}$$

where $H_{\boldsymbol{k}}$ is the hamiltonian of the system, which has the periodicity of the crystal lattice, and is a function of the momentum $\boldsymbol{k}$. For each $\boldsymbol{k}$ we have an equation 3.2 that gives a (infinite) set of solutions, which are labeled by the letter s.

The eigenvalues are the energies $E_{\boldsymbol{k},s}$ and the respective eigenvectors are the wavefunctions $\psi_{\boldsymbol{k},s}(\boldsymbol{r})$.

For a periodic material, the solutions to equation 3.2 are given by Bloch's theorem:

$$\psi_{\boldsymbol{k}s}(\boldsymbol{r}) = e^{i\boldsymbol{k}\cdot\boldsymbol{r}}u_{\boldsymbol{k}s}(\boldsymbol{r}) \tag{3.3}$$

with

$$u_{\boldsymbol{k}s}(\boldsymbol{r}) = u_{\boldsymbol{k}s}(\boldsymbol{r}+\boldsymbol{R}) \tag{3.4}$$

and $\boldsymbol{k}$ is a point within the Brillouin Zone (BZ) and s numbers the bands. $\boldsymbol{R}$ is the lattice vector.

$u_{\boldsymbol{k}s}(\boldsymbol{r})$ is called the Bloch factor and $e^{i\boldsymbol{k}\cdot\boldsymbol{r}}$ is called the Bloch phase factor.

Bands

In the previous subsection the solutions of the Schrödinger equation were ordered by a *band* index s. In practice, the eigenvectors and eigenvalues are ordered in increasing energy, so that $E_{\boldsymbol{k},0}$ is the lowest energy solution of equation 3.2, $E_{\boldsymbol{k},1}$ is the second lowest energy solution, and so on.

Actually, there are a number of property calculations where it is necessary to use the set of eigenvectors and eigenvalues that form a band as an entity in which mathematical operators must be applied (the gradient in momentum space, for instance). The Berry connection is an example. The Berry connection is defined as:

$$\boldsymbol{\xi}_{\boldsymbol{k}ss'} = i\langle u_{\boldsymbol{k}s}|\nabla_{\boldsymbol{k}}u_{\boldsymbol{k}s'}\rangle = \frac{i}{v_C}\int_{uc} d^3\boldsymbol{r}\, u^*_{\boldsymbol{k}s}(\boldsymbol{r})\nabla_{\boldsymbol{k}}u_{\boldsymbol{k}s'}(\boldsymbol{r}) \tag{3.5}$$

where v_C is the volume of the unit cell and uc stands for unit cell.

But the ordering (and grouping) of eigenstates by energy fails to give an usable entity for a gradient calculation, because it frequently has discontinuities, i.e. is non-analytic. Figure 3.2 ilustrates the problem. Where actual bands cross, the simple energy ordering cannot keep with the right band and jumps to another, leading in fact to a discontinuity.

In order to be able to apply a gradient in momentum space we need to have the eigenstates ordered and grouped in such a way that analycity is guaranteed. That is the

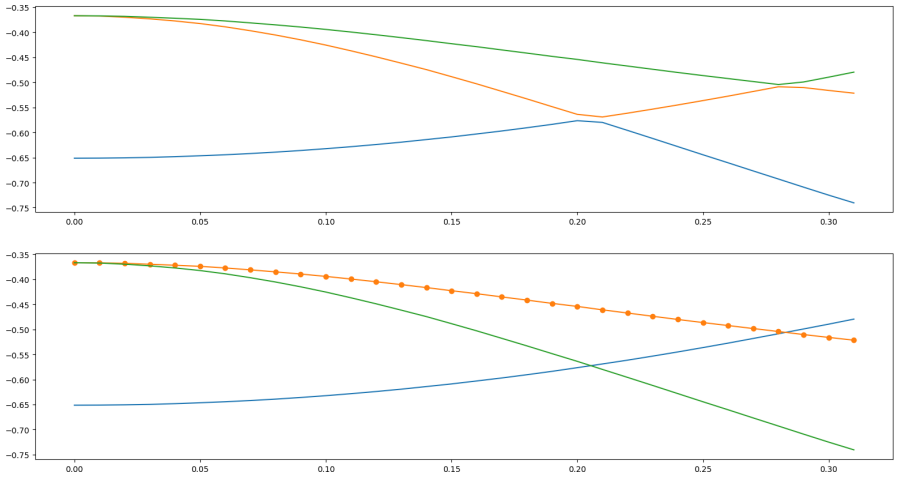


Figure 3.2: Example of electronic bands in one dimension. Different colors indicate different bands. Top: the bands according to the DFT output, that is, ordered by energy. Bottom: the bands according to continuity of the Bloch factors. The dots on the orange curve in the bottom figure indicate the positions of the k point sampling.

first goal of this suite of programs.

Berry curvatures can also be calculated using Equation 3.6 [1]

$$\mathbf{\Omega}_{s,s'} = i \int_{uc} d^3\boldsymbol{r} \, (\nabla_{\boldsymbol{k}} u^*_{\boldsymbol{k}s} \times \nabla_{\boldsymbol{k}} u_{\boldsymbol{k}s'}) \tag{3.6}$$

Band crossings

It is clear that the problem of classifying the eigenstates by bands where analycity applies is problematic at degeneracies i.e. band crossings.

There, any linear combination of eigenstates is also an eigenstate.

Lets consider two bands A and B, crossing at point k. The eigenstates of A are continuous at the left of k and at the right, and the same is true for eigenstates of B. But not necessarily at k. Supppose that ψ_A would be the wavefunction at k that would have continuity in band A and ψ_B for band B. Of course, ψ_A and ψ_B have the same eigenvalue.

But the numerical process of solving the Schrödinger equation at point k can give any result of the form

$$\psi_1 = a\psi_A + b\psi_B \tag{3.7}$$

$$\psi_2 = a'\psi_A + b'\psi_B \tag{3.8}$$

as long as ψ_1 and ψ_2 are orthogonal (the coeficcients have to be such that they assure orthogonality). Of course, these wavefunctions are not continuous with A or B, in general.

There are two ways to workaround this problem and restore analycity. One is to interpolate the wavefunction at k using the closest eigenstates of band A and then the ones of band B. The other is to apply an unitary transformation.

Unitary transformation The most frequent way of restoring analycity is to apply an unitary transformation

$$\begin{bmatrix} \psi_A \\ \psi_B \end{bmatrix} = U \begin{bmatrix} \psi_1 \\ \psi_2 \end{bmatrix} \tag{3.9}$$

where U is choosen in such a way as to 'iron out' the nonanalytic behaviour at k. In practice, this is done by minimizing the difference betwenn ψ_A and both ψ_A^- and ψ_A^+, and the same for ψ_B relative to ψ_B^- and ψ_B^+.

2 Graph Theory (GT)

Graph Theory (GT) studies the properties and the applications of a mathematical structure called a Graph. In a strict sense, a set of vertices (nodes) V and a set of edges E describes a Graph, $G(V, E)$. A vertex (node), $v \in V$, is a terminal point, and an edge, is a link between two nodes. This link must follow some relation, $R(v_1, v_2)$, between these nodes, for example, a co-authorship (edge) between two authors (nodes).

The edges are the set defined by

$$E = \{(v_1, v_2) : v_1, v_2 \in V \wedge \exists\, R(v_1, v_2)\} \tag{3.10}$$

The edges are oriented when the relationship between two nodes are diferent in each direction ($R(v_1, v_2) \neq R(v_2, v_1)$). Thus, the graph is a directed graph, for example, the papers' citations (one paper cites another, but the opposite is not true). However, if all pairs of nodes have the same edge in both directions ($R(v_1, v_2) = R(v_2, v_1)$) or the orientation is irrelevant, its graph is undirected.

Furthermore, all relations need not have the same weight. For example, getting back to the co-authorship graph, some authors may collaborate more times with a few than others. Thus, the edge between these two authors may include this weight $\omega(v_1, v_2)$ (it is a weighted graph). Nonetheless, if the weights ω_e of all edges e are the same ($\forall\, e \in E \;:\; \omega_e = \omega$) or the edges expresse a binary relation (may exist or not), the graph is unweighted. In this work, we will use unweighted and undirected graphs.

Another essential concept is that of the subgraph. A subgraph $g(V', E')$ is a graph formed by a subset of vertices $V' \subset V$ and edges $E' \subset E$ of a bigger graph $G(V, E)$.

A subgraph g, whose nodes do not have any edge between nodes of another subgraph $g', g \neq g'$ is called a component.

Graph theory is very well studied, and there are many implemented algorithms to study graphs' properties. We will use the *networkx* python module for the implementation and analysis of graphs.

3 Unsupervised Machine Learning (UML)

AI solves problems that normally require a human to make decisions. The cornerstone of Artificial Intelligence is designing agents that learn how to make informed decisions in possibly ever-changing environments without human supervision. Machine learning is a subset of AI where models learn patterns from datasets. How an artificial agent learns these patterns classifies them into Supervised or Unsupervised machine learning (UML) models.

Unlike supervised machine learning, which uses data labeled by an expert and learned patterns from comparison, UML algorithms are used for clustering or association because they identify patterns from uncategorized data. These algorithms recognize patterns in the datasets using some metrics to evaluate differences or similarities between data.

UML algorithms for clustering use a dataset of uncategorized points, and a problem dependent metric. The algorithm processes the points to form groups (clusters) represented by some pattern or structure determined by the metric. The algorithm is called exclusive when a datapoint belongs exclusively to a single cluster. The band-crossing problem lies in this class of algorithms due to its structure. Thus, we will work with this type of clustering algorithm.

4 Optical conductivity and Second Harmonic Generation

For the calculation of the linear optical conductivity and the second harmonic generation, we use the length gauge description of the reduced density matrix [2–4], as described in reference [4].

For the case of a cold d dimensional semiconductor, the linear conductivity can be expressed in terms of a single, interband, contribution, given by Equation 3.11

$$\sigma^{\beta\alpha}(\omega) = \frac{ie^2}{\hbar}\sum_{s's}\int_{BZ}\frac{d^d\boldsymbol{k}}{(2\pi)^d}\frac{\Delta\epsilon_{\boldsymbol{k}s's}\xi^{\beta}_{\boldsymbol{k}ss'}\xi^{\alpha}_{\boldsymbol{k}s's}}{(\hbar\omega+i\Gamma)-\Delta\epsilon_{\boldsymbol{k}s's}}\Delta f_{\boldsymbol{k}ss'}, \tag{3.11}$$

where $\Delta\epsilon_{\boldsymbol{k}s's} = \epsilon_{\boldsymbol{k}s'} - \epsilon_{\boldsymbol{k}s}$, $\Delta f_{\boldsymbol{k}ss'} = f_{\boldsymbol{k}s} - f_{\boldsymbol{k}s'}$, $\epsilon_{\boldsymbol{k}s}$ is the eigenvalue of band s at $\boldsymbol{k}$, and $f_{\boldsymbol{k}s} = f(\epsilon_{\boldsymbol{k}s})$ is the Fermi-Dirac distribution function. Note that with the knowledge of the band energies, $\epsilon_{\boldsymbol{k}s}$, and the Berry connections, $\xi^{\beta}_{\boldsymbol{k}ss'}$, both of which can be determined by a DFT calculation, we can compute the optical response of a crystal. The indexes α and β run through the spatial coordinates x, y and z (depending on the material's dimension).

The second order response can be calculated for the case of the second harmonic generation. Following the procedure described in ref. [4] we obtain, for a diagonal component

of the SHG, $\sigma^{\beta\alpha_1\alpha_2}(\omega,\omega)$, Eq.(3.12).

$$\sigma^{\beta\alpha_1\alpha_2}(\omega) = -\frac{e^3}{\hbar}\int\limits_{BZ}\frac{d^2\boldsymbol{k}}{(2\pi)^2}\left[\sum_{s'\neq s}\frac{1}{2}\frac{\Delta\epsilon_{\boldsymbol{k}ss'}}{g_{\omega\boldsymbol{k}ss'}}\left\{(\nabla^{\alpha_2}\Delta\epsilon_{\boldsymbol{k}ss'})\left[\xi^{\beta}_{\boldsymbol{k}s's},\xi^{\alpha_1}_{\boldsymbol{k}ss'}\right]\frac{\Delta f_{\boldsymbol{k}s's}}{h^2_{\omega\boldsymbol{k}ss'}}+\left[\xi^{\beta}_{\boldsymbol{k}s's},(\xi^{\alpha_1}_{\boldsymbol{k}ss'})_{;\alpha_2}\right]\frac{\Delta f_{\boldsymbol{k}s's}}{h_{\omega\boldsymbol{k}ss'}}\right\}\right. \tag{3.12}$$

$$-\frac{i}{4}\sum_{s'\neq s\neq r}\frac{\Delta\epsilon_{\boldsymbol{k}ss'}}{g_{\omega\boldsymbol{k}ss'}}\left\{\frac{\Delta f_{\boldsymbol{k}s'r}}{h_{\omega\boldsymbol{k}rs'}}\left(\xi^{\beta}_{\boldsymbol{k}s's}\xi^{\alpha_2}_{\boldsymbol{k}sr}\xi^{\alpha_1}_{\boldsymbol{k}rs'}+\xi^{\alpha_1}_{\boldsymbol{k}s'r}\xi^{\alpha_2}_{\boldsymbol{k}rs}\xi^{\beta}_{\boldsymbol{k}ss'}\right)-\frac{\Delta f_{\boldsymbol{k}rs}}{h_{\omega\boldsymbol{k}sr}}\left(\xi^{\beta}_{\boldsymbol{k}s's}\xi^{\alpha_1}_{\boldsymbol{k}sr}\xi^{\alpha_2}_{\boldsymbol{k}rs'}+\xi^{\alpha_2}_{\boldsymbol{k}s'r}\xi^{\alpha_1}_{\boldsymbol{k}rs}\xi^{\beta}_{\boldsymbol{k}ss'}\right)\right\}$$

$$\left.+(\alpha_1\leftrightarrow\alpha_2)\right]$$

where

$$g_{\omega\boldsymbol{k}ss'} = 2(\hbar\omega+i\Gamma)-\Delta\epsilon_{\boldsymbol{k}ss'} \tag{3.13}$$

$$h_{\omega\boldsymbol{k}ss'} = (\hbar\omega+i\Gamma)-\Delta\epsilon_{\boldsymbol{k}ss'} \tag{3.14}$$

$$\left[\xi^{\beta}_{\boldsymbol{k}s's},\xi^{\alpha_1}_{\boldsymbol{k}ss'}\right] = \xi^{\beta}_{\boldsymbol{k}s's}\xi^{\alpha_1}_{\boldsymbol{k}ss'}-\xi^{\alpha_1}_{\boldsymbol{k}s's}\xi^{\beta}_{\boldsymbol{k}ss'} \tag{3.15}$$

$$(\xi^{\alpha_1}_{\boldsymbol{k}ss'})_{;\alpha_2} = (\nabla^{\alpha_2}\xi^{\alpha_1}_{\boldsymbol{k}ss'})-i\left(\xi^{\alpha_2}_{\boldsymbol{k}ss}-\xi^{\alpha_2}_{\boldsymbol{k}s's'}\right)\xi^{\alpha_1}_{\boldsymbol{k}ss'} \tag{3.16}$$

Workflow

1 Preprocessing

After creating the files and directories mentioned in section Running, the next thing is to run the script `preprocess`,

```
berry preprocess input_file [script options]
```

which will do the following:

1. Reads file `inputfile` with the description of the run wanted.
2. Creates directories `log` and `data`, and inside `data` creates directories `wfc` and `geometry`.
3. Runs the scf calculation using Quantum Espresso, if it has not ran before.
4. Generates an nscf input file according to the input file and runs an nscf calculation, using Quantum Espresso, if it has not ran before.
5. Reads data from the nscf calculation and saves to a file.
6. Makes several simple calculations that will be used afterwards in other scripts, and save them to several files.

An input file for the berry run has to be created in the working directory with several parameters. The minimum parameters needed are:

- origin of k-points (k0)
- number of k-points in each direction (nkx, nky, nkz)
- step of the k-points (step)
- number of bands to be calculated (nbnd)

In the following section a detailed explanation of the input file is given.

The script `preprocess` creates the following files in the `data` directory:

- *phase.npy*
- *neighbors.npy*
- *eigenvalues.npy*
- *occupations.npy*
- *positions.npy*
- *kpoints.npy*
- *nktoijl.npy*
- *ijltonk.npy*

and *datafile.npy* which is the main data file.

File *neighbors.npy* lists the neighbors of each k-point (which neighbors to consider may be an input parameter in the future).

File *phase.npy* saves for each (k,r) the phase factor of the Bloch functions.

This finishes the preparatory phase. The options for `preprocess` are shown in table 4.1.

Table 4.1: Options for script `preprocess`

`-h, --help`	Show a help message and exit
`-o file_path`	Name of output log file Extension will be ignored
`-flush`	Flush the ouput to stdout
`-v`	Increase output verbosity

All log files are created in directory `log`.

2 The input file

The input file read by the `preprocess` script defines several important aspects of the following scripts, and it is important to have it right.

An example of the minimum input file is shown in figure 4.1. A full list of the parameters that are accepted with the respective defaults (if any) is shown in table 4.2. There is a unique reference number that is attributed when running `preprocess` to guarantee consistency, It can be changed in the input file, but is not recomendable, because it is too easy to forget it and attribute the same value for different runs.

```
k0 0.00 0.00 0.00
nkx 100
nky 100
nkz 1
step 0.005
nbnd 8
```

Figure 4.1: Example of input file for `preprocess`.

2.1 The sampling of the Brillouin Zone

In principle, one wants to calculate the bands in the full Brillouin Zone (BZ) for the calculation of the linear and second harmonic condutivity. One can always sample just a part of it, of course.

For computational reasons, in version 2.0 (as in version 1.0) the sample has to be a continuous line in 1D materials, a rectangle in 2D materials and a parallelepiped in 3D materials. It is in the input file that these are defined.

The first variable (mandatory) is named `k0` and it is the origin in k-space where the sampling will start. It is a set of three real numbers. Figure 4.1 shows an example, where the Γ point is chosen.

A set of three vectors can be defined indicating the orientation of the sample volume (or two vectors for the orientation in a 2D material). These vectors are optional, and if not included in the input file, the default is that orientation 1 is along the x axis, orientation 2 is along the y axis and orientation 3 is along the z axis. The variables that define the three vectors are `kvector1`, `kvector2` and `kvector3`, each taking three real numbers, as the variable `k0`.

The number of points in each direction that will be calculated are in variables `nkx`, `nky` and `nkz` (mandatory). They should be chosen so that they cover the volume (or line or area) wanted, taking into account the variable `step` (also mandatory).

For an one dimension material, `nky = nkz = 1`, so the material has to be align with the x axis. For a two dimensions material, it has to be in the plane xy, so `nkz = 1`. This is how the program knows the number of dimensions to be used.

It is strongly suggested that a preliminary scf run is performed to ascertain the correct lattice and reciprocal lattice vectors and dimensions, and so determine the parameters to be used.

For reference, we show below two possible samplings that for a 2D hexagonal material and a 3D FCC material.

2D hexagonal material Let's consider a material with the following lattice and reciprocal lattice vectors (see figure 4.2):

Real	Reciprocal
$\mathbf{a}_1 = \frac{\sqrt{3}a}{2}\mathbf{e}_x - \frac{a}{2}\mathbf{e}_y$	$\mathbf{b}_1 = \frac{2\pi}{a\sqrt{3}}\mathbf{k}_x - \frac{2\pi}{a}\mathbf{k}_y$
$\mathbf{a}_2 = \frac{\sqrt{3}a}{2}\mathbf{e}_x + \frac{a}{2}\mathbf{e}_y$	$\mathbf{b}_2 = \frac{2\pi}{a\sqrt{3}}\mathbf{k}_x + \frac{2\pi}{a}\mathbf{k}_y$

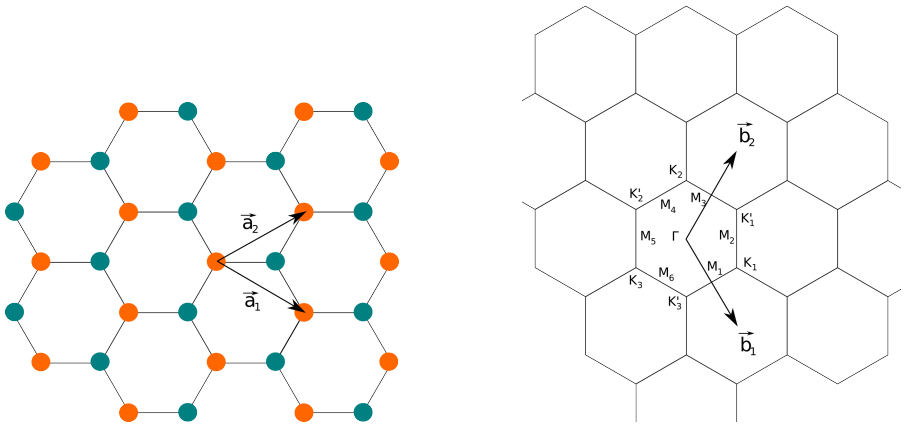


Figure 4.2: 2D hexagonal lattice with the lattice vectors (left) and the corresponding reciprocal space lattice and vectors (right).

To sample the area equivalent to the BZ using a rectangle, one can chose the one shown in figure 4.3.

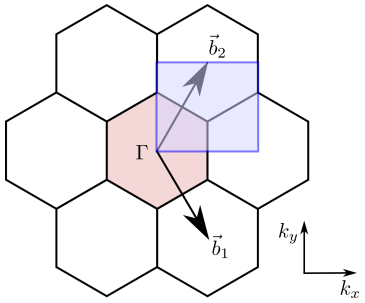


Figure 4.3: Rectangle with an equivalente area to the BZ for a 2D hexagonal lattice.

It can be seen that the side along x has a size

$$k_x = |\mathbf{b}_1 + \mathbf{b}_2| = \frac{4\pi}{a\sqrt{3}} \tag{4.1}$$

and along the y axis is simply the y component of $\mathbf{b}_2$:

$$k_y = \frac{2\pi}{a} \tag{4.2}$$

If we call Δk the step (it has to be the same for both directions) then we have

$$n_1 = \frac{k_x}{\Delta k} = \frac{4\pi}{a\sqrt{3}\Delta k}$$
$$n_2 = \frac{k_y}{\Delta k} = \frac{2\pi}{a\Delta k}$$

n_1 and n_2 have to be integers, and since we don't want to have repeated points in reciprocal space, these numbers have to be rounded by truncation.

We also have the important relation:

$$n_1 = \frac{2}{\sqrt{3}} n_2 \tag{4.3}$$

FCC 3D crystal Let's consider a material with the following FCC lattice and reciprocal lattice vectors:

Real	Reciprocal
$\mathbf{a}_1 = \frac{a}{2}(\mathbf{e}_y + \mathbf{e}_z)$	$\mathbf{b}_1 = \frac{2\pi}{a}(-\mathbf{k}_x + \mathbf{k}_y + \mathbf{k}_z)$
$\mathbf{a}_2 = \frac{a}{2}(\mathbf{e}_x + \mathbf{e}_z)$	$\mathbf{b}_2 = \frac{2\pi}{a}(\mathbf{k}_x - \mathbf{k}_y + \mathbf{k}_z)$
$\mathbf{a}_3 = \frac{a}{2}(\mathbf{e}_x + \mathbf{e}_y)$	$\mathbf{b}_3 = \frac{2\pi}{a}(\mathbf{k}_x + \mathbf{k}_y - \mathbf{k}_z)$

One possible choice of sample is to chose the following three directions in k-space:

$$\hat{k}_1 = (0, 0, 1)$$
$$\hat{k}_2 = \frac{1}{\sqrt{2}}(1, -1, 0)$$
$$\hat{k}_3 = \frac{1}{\sqrt{2}}(1, 1, 0)$$

with sizes:

$$|\mathbf{k}_1| = \frac{4\pi}{a}$$
$$|\mathbf{k}_{2,3}| = \frac{2\pi}{a}\sqrt{2}$$

Which means that the number of points sampling in directions 2 and 3 should be the same:

$$n_2 = n_3 \tag{4.4}$$

and

$$n_1 = \frac{n_2}{\sqrt{2}}$$

Another way of seeing this is:

$$\mathbf{k}_1 = \mathbf{b}_1 + \mathbf{b}_2$$
$$\mathbf{k}_2 = \frac{1}{2}(\mathbf{b}_2 - \mathbf{b}_1)$$
$$\mathbf{k}_3 = \frac{1}{2}(\mathbf{b}_1 + \mathbf{b}_2) + \mathbf{b}_3$$

2.2 The bands to be used

It is necessary to specify the set of bands that will be processed. It is important to obtain and look at a normal band diagram of the material before chosing what bands to include. Otherwise, a large amount of time and computer resources may be spent uselessly.

The parameter `nbnd` (mandatory) gives the number of bands to be calculated. This number will be used in the nscf calculation of QE. This also defines the highest band that will be used, that will have a band number of `nbnd`-1.

Ideally, this should be the highest band before a gap in the bands. If it is not, there will be incomplete bands in the final result of the berry calculation, and they will be unusable.

There is another optional parameter `wfcut` that allows us to exclude some of the lowest energy bands from the berry calculations (not from the QE calculations, of course). It is the number of the highest band to be excluded (band numbering starts at 0 for the lowest band, and goes up to `nbnd`-1). So, if `wfcut` is negative, all bands will be included.

It is important that there is a gap between the excluded bands and the included ones. Otherwise, there will likely be incomplete (and so useless) bands.

2.3 Removing volume from the supercell

When performing 1D and 2D calculations, the wavefunctions provided by the DFT don't fill the whole supercell. There are regions in the supercell where the wavefunctions should be zero, but actually there is some noise. Knowing this, it should be possible to remove part of the supercell, saving computational resources and without compromising the results.

In order to do this, the parameters `z1` and `deltaz` and/or `y1` and `deltay` must be included in the input file. `z1` and `y1` are used to define the initial limit of the region to be removed, and should have a value between 0 and a_3 and a_2, respectively. Then `deltaz` and `deltay` define the width of the region to be removed, and should have a value between 0 and a_3 and a_2, respectively. The default values are 0, meaning no volume is removed. The final limit, `z2` and `y2` is then calculated by one of the following expressions:

$$z_2 = \begin{cases} z_1 + \Delta z, & \text{if } z_1 + \Delta z \leq a_3 \\ z_1 + \Delta z - a_3, & otherwise \end{cases}$$

$$y_2 = \begin{cases} y_1 + \Delta y, & \text{if } y_1 + \Delta y \leq a_2 \\ y_1 + \Delta y - a_2, & otherwise \end{cases}$$

Each of the cases will remove volume from the supercell either from the middle (see figure or from its the sides (see figure 4.4).

(a) (b)

Figure 4.4: When $y_1 \leq a_2$, () the region removed is located on the center of the supercell. Otherwise, (a) the region removed will be splitted into two sides.

Table 4.2: List of variables for the input file of `preprocess`.

Role of keyword	**Variable (and defaults)**
origin of k-points (3 real numbers)	`k0`
number of k-points in each direction (integer)	`nkx, nky, nkz`
step of the k-points (real number)	`step`
number of bands to be calculated (integer)	`nbnd`
number of processors	`npr = 1`
dft directory	`dftdirectory = 'dft/'`
scf file name	`name_scf = 'scf.in'`
directory where wavefunctions will be saved	`wfcdirectory = 'wfc/'`
point in real space where all phases match	`point = 1.178097`
software for DFT	`program = 'QE'`
unique reference of run	`refname` = date and time
first direction in k-space	`kvector1 = 1.0 0.0 0.0`
second direction in k-space	`kvector2 = 0.0 1.0 0.0`
third direction in k-space	`kvector3 = 0.0 0.0 1.0`
last band NOT accounted for	`wfcut = -1`
initial limit of volume removal	`y1, z1 = 0`
width of volume removal region	`deltay, deltaz = 0`

3 Extraction of wavefunctions

The next step is to extract the wavefunctions from the format used in the DFT package (in this case it is Quantum Espresso), make them coherent and save them in a format adequate for the **berry** suite.

The script that does this job is `wfcgen`:

```
berry wfcgen [script options]
```

It uses the package wfck2r.x of the Quantum Espresso suite to extract and save the wavefunctions from the original format to a numpy file, in position space, one for each k-point and band. It uses the same number of processes as used before in the DFT calculations. Since wfck2r.x extracts $u_{\mathbf{ks}}$, that's what is saved.

After extracting the wavefunctions, a point in position space is choosen (see table 4.2), away from points of high symmetry, and all wavefunctions are multiplied by a phase such that at that point all have zero phase (are real).

The wavefunctions are then saved in the directory choosen to save them (see table 4.2; default is *working_directory/wfc/*), and the files will be named in the form *k0**b0XX.wfc* where **** is replaced by the number of the k point and *XX* is replaced by the number of the band.

Bands are as given by the DFT software i.e. ordered by energy, and the lowest energy band is numbered zero and so on.

The options for `wfcgen` are shown in table 4.3. The default is to generate all formated wavefunctions, but for debugging it may be useful to generate just for a single k-point or a single wavefunction, for which we have the options `-nk` and `-band` (has to be used in conjunction with `-nk`).

Table 4.3: Options for script `wfcgen`

`-nk`	Generates wavefunctions for a single k-point (default: All)
`-band`	Generates wavefunctions for a single k-point and band (default: All)
`-h`, `--help`	Show a help message and exit
`-o file_path`	Name of output log file Extension will be ignored
`-flush`	Flush the ouput to stdout
`-v`	Increase output verbosity

4 Degenerate subspace basis rotation

After the wavefunctions have been extracted by `wfcgen`, and before the dot products are computed by `dot`, it is possible to run the optional script `degenrot`.

```
berry degenrot [script options]
```

This script is optional but recommended whenever the band structure contains degenerate points, i.e. k-points at which two or more bands share the same (or numerically indistinguishable) energy eigenvalue. Without it, the arbitrary basis chosen by the DFT (Density Functional Theory) solver at those k-points propagates into the dot products and can degrade the quality of the subsequent band classification by `cluster`.

The problem of degenerate subspaces

At a generic k-point in the Brillouin Zone (BZ), the eigenstates of the Kohn-Sham Hamiltonian (see equation 3.2) are non-degenerate and uniquely determined up to an overall complex phase. However, at high-symmetry points, or along lines of high symmetry, two or more bands can share the same eigenvalue $E_{\boldsymbol{k}}$. In that case, any linear combination of the corresponding eigenstates is also an eigenstate with the same energy. The DFT solver (here Quantum Espresso) returns an arbitrary orthonormal basis for this degenerate subspace, which in general bears no relation to the basis chosen at neighbouring k-points.

As a consequence the Bloch factors $u_{\boldsymbol{k}s}(\boldsymbol{r})$, which should vary smoothly with $\boldsymbol{k}$ within each band s, can have an arbitrary orientation at every degenerate k-point. This discontinuity manifests as anomalously small dot products $|\langle u_{\boldsymbol{k}s}|u_{\boldsymbol{k}'s}\rangle|$ between a degenerate k-point and its neighbours, even when the two k-points belong to the same band.

The goal of `degenrot` is to rotate the wavefunction basis at every degenerate k-point so that it aligns as closely as possible with the basis of the surrounding non-degenerate region, restoring continuity before the dot products are computed.

Algorithm

The algorithm proceeds in four phases.

Phase 1 — Detection of degenerate subspaces. For every k-point $\boldsymbol{k}$ and every pair of bands (s, s') with $s < s'$, the script checks whether

$$|E_{\boldsymbol{k},s} - E_{\boldsymbol{k},s'}| < \varepsilon \tag{4.5}$$

where ε is the energy threshold (default 10^{-4}, in the same units as the eigenvalues). A union-find algorithm [10] groups all mutually-degenerate bands at each k-point into a degenerate subspace of dimension $N \geq 2$.

Phase 2 — Formation of degenerate zones. For each unique band-group signature (i.e. each distinct frozenset of band indices), the set of k-points that share that degeneracy is collected and its connected components are found by breadth-first search (BFS) over the neighbour graph from *neighbors.npy*.

Each connected component is called a *degenerate zone*: a contiguous region of k-space in which the same N bands are degenerate. A single calculation may contain many independent zones, each treated separately.

Phase 3 — Reference selection. To define an unambiguous target basis for a zone, `degenrot` searches for a *boundary k-point*: a k-point that belongs to the zone and has at least one neighbour outside the zone where the N bands are *non*-degenerate (i.e. where the minimum pairwise energy gap exceeds ε). Among all such boundary–neighbour pairs the one with the largest minimum pairwise energy separation is chosen as the reference, because it provides the most unambiguous basis.

If no non-degenerate external neighbour exists (the zone is *fully isolated*), an arbitrary k-point in the zone is chosen as the root. A warning is written to the log file and the zone is given an internally consistent basis, but the absolute orientation of that basis is arbitrary.

Phase 4 — Propagation of the basis rotation by BFS. Starting from the boundary reference k-point, the script propagates the rotation outward through the zone via BFS. At each k-point $\boldsymbol{k}$ (with BFS parent $\boldsymbol{k}_{\text{ref}}$, which has already been rotated):

1. The $N \times N$ Bloch-phase-corrected overlap matrix is computed:

$$M_{ij} = \langle u_i^{\text{ref}} | u_j^{\text{cur}} \rangle = \frac{1}{N_r} \sum_{\boldsymbol{r}} e^{i(\boldsymbol{k}-\boldsymbol{k}_{\text{ref}})\cdot\boldsymbol{r}}\, u^*_{\boldsymbol{k}_{\text{ref}},i}(\boldsymbol{r})\, u_{\boldsymbol{k},j}(\boldsymbol{r}) \tag{4.6}$$

where N_r is the number of real-space grid points, the sum runs over all grid points $\boldsymbol{r}$, and $i, j = 1, \ldots, N$ index the bands in the degenerate subspace.

For non-colinear (NC) calculations, each Bloch factor is a two-component spinor $u_{\boldsymbol{k}s} = (u^{\uparrow}_{\boldsymbol{k}s}, u^{\downarrow}_{\boldsymbol{k}s})$ and the inner product sums over both spin components:

$$M_{ij} = \frac{1}{N_r} \sum_{\boldsymbol{r}} e^{i(\boldsymbol{k}-\boldsymbol{k}_{\text{ref}})\cdot\boldsymbol{r}} \left[u^{*\uparrow}_{\boldsymbol{k}_{\text{ref}},i}(\boldsymbol{r})\, u^{\uparrow}_{\boldsymbol{k},j}(\boldsymbol{r}) + u^{*\downarrow}_{\boldsymbol{k}_{\text{ref}},i}(\boldsymbol{r})\, u^{\downarrow}_{\boldsymbol{k},j}(\boldsymbol{r}) \right] \tag{4.7}$$

2. The Singular Value Decomposition (SVD) [9] of M is computed:

$$M = U\, \Sigma\, V^{\dagger} \tag{4.8}$$

where U and V are $N \times N$ unitary matrices and $\Sigma = \text{diag}(\sigma_1, \sigma_2, \ldots, \sigma_N)$ with $\sigma_i \geq 0$.

3. The optimal rotation matrix is obtained from the orthogonal Procrustes problem [7]: find the $N \times N$ unitary matrix R that maximises $\text{Re}\,\text{Tr}(MR)$. The exact solution is

$$R = V\, U^{\dagger} \tag{4.9}$$

4. The new Bloch factors at $\boldsymbol{k}$ are

$$u^{\text{new}}_{\boldsymbol{k},j}(\boldsymbol{r}) = \sum_{i=1}^{N} R_{ij}\, u_{\boldsymbol{k},i}(\boldsymbol{r}), \qquad j = 1, \ldots, N \tag{4.10}$$

and they overwrite the corresponding *.wfc* files in-place. For NC calculations the same matrix R is applied to each spin component independently.

Singular values as a quality measure

The singular values σ_i of the overlap matrix M are the cosines of the *principal angles* [8] between the N-dimensional subspace spanned by the reference basis and the N-dimensional subspace spanned by the current basis:

$$\sigma_i = \cos\theta_i, \qquad i = 1, \ldots, N \tag{4.11}$$

where $0 \leq \theta_1 \leq \theta_2 \leq \ldots \leq \theta_N \leq \pi/2$ are the principal angles. If both bases span exactly the same subspace (as is the case when the degeneracy is clean and the k-spacing

is small), all singular values equal one: the two subspaces coincide and the Procrustes rotation is exact.

A singular value significantly below one indicates that one direction in the current subspace is tilted away from the reference subspace, which means the degenerate bands are mixing with states outside the chosen N-dimensional model. The minimum singular value $\min_i \sigma_i$ is the most sensitive diagnostic: it measures the worst-case alignment across all N directions. Typical values above 0.99 indicate a reliable rotation; values below 0.9 should prompt the user to check whether additional bands ought to be included in the degenerate group.

Note that a permutation of the basis vectors, a sign flip, or any other unitary rearrangement of the current basis *within* the same subspace gives singular values all equal to one, because the overlap matrix M is itself unitary in that case. A small singular value therefore strictly means the two subspaces are genuinely different, not merely expressed in different bases.

Boundary consistency check

After the BFS rotation has been applied to all k-points in the zone, the script checks whether the resulting gauge is *globally consistent* across the zone boundary. For every zone k-point $\boldsymbol{k}$ that has a non-degenerate external neighbour $\boldsymbol{k}'$ (i.e. a grid neighbour outside the zone where the N bands are split), the minimum singular value $\sigma_{\min}(\boldsymbol{k}, \boldsymbol{k}')$ is computed from the overlap matrix between the rotated basis at $\boldsymbol{k}$ and the unmodified basis at $\boldsymbol{k}'$.

The *boundary spread*

$$\Delta\sigma = \max_{\text{boundary edges}} \sigma_{\min} - \min_{\text{boundary edges}} \sigma_{\min} \tag{4.12}$$

measures how uniformly the rotated zone aligns with its external neighbourhood. A large spread indicates that the BFS tree imposed a path-dependent gauge choice, so that different boundary k-points align well or poorly depending on which BFS path was used to reach them.

An important invariant: boundary singular values are *gauge-invariant*. Iterative refinement cannot reduce the spread; the holonomy correction described below must act before refinement when the spread is large.

Gauge repair — holonomy correction

Trigger: boundary spread > 0.1 and the zone contains at least `-holonomy-min-plaquettes` internal plaquettes (default: 2). When the spread is very large (> 0.5) and the worst boundary singular value is below 0.6 — indicating a topological discontinuity — the correction is skipped and a warning is logged, because the holonomy cannot be reduced in

that case (see below).

Physical meaning: The BFS tree forced a path-dependent gauge choice. No topological obstruction prevents a smooth consistent gauge, but the BFS did not find it.

Algorithm: The script walks all *elementary plaquettes*, i.e. minimal 4-cycles $\boldsymbol{k}_0 \to \boldsymbol{k}_1 \to \boldsymbol{k}_2 \leftarrow \boldsymbol{k}_3 \leftarrow \boldsymbol{k}_0$ in the zone graph, and computes for each plaquette the *holonomy*

$$H = R_{01}\, R_{12}\, R_{23}^{\dagger}\, R_{30}^{\dagger} \tag{4.13}$$

where each R_{ij} is the Procrustes rotation of equation (4.9) evaluated between the current Bloch factors at $\boldsymbol{k}_i$ and $\boldsymbol{k}_j$. A perfectly consistent gauge gives $H = \mathbf{1}$; the departure from identity is the *loop defect*.

The defect is distributed by applying fractional corrections $H^{-1/4}$, $H^{-1/2}$, $H^{-3/4}$ to the three non-anchor vertices of the plaquette. The fractional power is computed via two successive matrix square roots (using `scipy.linalg.sqrtm`) followed by projection onto U(N). The procedure iterates over all plaquettes until the maximum holonomy defect $\|H - \mathbf{1}\|_F$ falls below a tolerance (default 10^{-4}) or a maximum number of sweeps is reached (default 20).

Early exit on topological holonomy: If the maximum defect does not decrease between two consecutive sweeps (relative change $< 10^{-8}$), the zone carries an *irreducible* Berry-phase holonomy: the total holonomy around the zone boundary is a topological invariant that cannot be eliminated by local plaquette corrections. The correction exits immediately with a warning; iterative refinement proceeds but may not fully converge.

Effect: The internal gauge consistency improves, giving the subsequent iterative refinement a better starting point and allowing it to converge more quickly. The boundary spread (being gauge-invariant) is not changed by this step.

Iterative gauge refinement

After any gauge repair (or directly if the spread is small), all zone k-points are refined simultaneously by a multi-neighbour Procrustes sweep.

Each sweep (a Gauss-Seidel pass, or a Jacobi pass when parallelism is enabled) re-rotates every zone k-point to best align with *all* its valid neighbours simultaneously: both zone-internal neighbours (also being refined) and external non-degenerate neighbours (fixed anchors that prevent global gauge drift). The overlap matrices from every neighbour are summed before the Procrustes step so all neighbours vote on the rotation:

$$M_{\text{total}}(\boldsymbol{k}) = \sum_{\boldsymbol{k}' \in \mathcal{N}(\boldsymbol{k})} M(\boldsymbol{k}', \boldsymbol{k}) \tag{4.14}$$

where $\mathcal{N}(\boldsymbol{k})$ includes all zone-internal neighbours and all non-degenerate external neighbours.

Convergence is declared when $\max_{\boldsymbol{k}} \|R_{\boldsymbol{k}} - \mathbf{1}\|_F$ falls below the refinement tolerance (default 10^{-4}) or the maximum iteration count is reached. The iteration limit scales with zone size: $\max(N_{\text{base}},\, N_{\text{base}} + 3\,|\text{zone}|)$, capped at a hard maximum (defaults: base 50, cap 500).

Stagnation early exit. If the residual is still more than ten times the tolerance but has improved by less than a factor of two over the last 30 iterations, the solver has reached a topological floor and further iterations are wasteful. Refinement exits early with a warning; the final rotations are still written and the zone is marked in the log.

Anderson acceleration. Each Gauss-Seidel sweep is followed by an Anderson mixing step [11] that fits a least-squares linear combination of the last m sweep residuals (default $m = 10$) to predict the fixed point and extrapolates the rotation matrices in that direction. Each $N \times N$ block of the extrapolated flat state vector is projected back onto $\mathrm{U}(N)$ via SVD. The safety criterion rejects any Anderson step that would exceed five times the plain Gauss-Seidel step. This acceleration converts the slow linear convergence ($\rho \approx 0.97$ per iteration without acceleration) into effective superlinear convergence, typically reducing the required iteration count by a factor of 5–10.

Jacobi parallel sweep. When the `np` option is greater than one and the in-memory wavefunction cache is active, the Gauss-Seidel sweep is replaced by a *Jacobi* parallel sweep: a snapshot of the current wavefunction state is taken at the start of each sweep, all k-points compute their rotation independently (using the snapshot), and the updates are applied after all workers have finished. This parallelises the most expensive part of the refinement at the cost of approximately twice as many iterations to converge (Jacobi converges at $\rho \approx 0.97$ vs. $\rho^2 \approx 0.94$ for Gauss-Seidel). The break-even is four or more workers; beyond that Jacobi gives a net speedup.

Parallelism

`degenrot` supports three levels of parallelism, all controlled by the single `np` option:

1. **Phase 1 (k-point detection):** the degeneracy check for each k-point is independent and dispatched to a process pool (`ProcessPoolExecutor`).

2. **Zone-level:** multiple degenerate zones that share the same band signature (and thus have disjoint k-point sets) are processed simultaneously using separate processes with copy-on-write memory sharing (fork start method; Linux and macOS only).

3. **Sweep-level (Jacobi):** within a single zone's iterative refinement, the per-k-point rotation computation is parallelised using a thread pool (`ThreadPoolExecutor`); requires the in-memory wavefunction cache to be active.

These three levels are mutually compatible and engage automatically based on the available zone structure and cache state.

In-memory wavefunction cache

By default, `degenrot` pre-loads all wavefunction files for a zone (zone k-points plus their grid neighbours) into a RAM dictionary before starting the boundary check and iterative refinement. All subsequent reads come from RAM instead of disk, which can reduce I/O by a factor of 100 or more for large zones.

When the cache is active, *all writes during holonomy correction and iterative refinement are deferred to RAM only.* A single disk flush at the end of each zone replaces the thousands of per-iteration file writes that would otherwise occur. For a zone of 1600 k-points and two non-collinear bands this reduces disk traffic per zone from tens of gigabytes (one full write per iteration) to a single pass of roughly 35 GB written once.

Available RAM is queried via `psutil` (a required dependency). The cache is skipped automatically when its estimated footprint would exceed 80% of available RAM; a conservative 4 GB hard limit applies only if `psutil` fails at runtime.

Output

`degenrot` overwrites the *.wfc* files of the rotated k-points in-place and saves one additional file *degenzones.npy* in the *data* directory. Each row of *degenzones.npy* has the form

```
[zone_id,  b_1, b_2, ..., b_N,  k-point]
```

where `zone_id` identifies the degenerate zone, $b_1 \dots b_N$ are the absolute band indices of the degenerate subspace (padded with -1 if the subspace dimension is smaller than the maximum across all zones), and `k-point` is the index of the k-point.

A log file is written to *log/degenrotation.log* (or the name given with `-o`). The final summary table records, for every zone, the bands involved, the type of reference (*boundary* or *isolated*), the total number of k-points in the zone, the number of k-points rotated in Phase 4, the BFS root k-point, the mean BFS singular value, and the number of holonomy correction sweeps performed (column *Holo*).

The options for `degenrot` are shown in table 4.4.

5 Dot product

`dot` reads the wavefunctions saved in directory *wfc*, removes the phase factor of the Bloch functions, and calculates the dot product of the Bloch factors of the neighboring k points.

```
berry dot [script options]
```

The neighboring k points are the four nearest neighbors, as determined in the preprocessing and saved in file *neighbors.npy*.

Table 4.4: Options for script `degenrot`.

`-h, --help`	Show a help message and exit
`-ethr`	Energy threshold for degeneracy detection in eigenvalue units (default: 10^{-4})
`-c`	Compress the output *.wfc* files
`-max-ref-iter`	Minimum refinement iterations per zone (default: 50)
`-ref-iter-cap`	Hard cap on refinement iterations (default: 500)
`-anderson-m`	Anderson mixing history window; 0 to disable (default: 10)
`-ref-tol`	Refinement convergence tolerance (default: 10^{-4})
`-no-holonomy`	Disable holonomy correction
`-holonomy-max-iter`	Maximum holonomy correction sweeps (default: 20)
`-holonomy-tol`	Holonomy convergence tolerance (default: 10^{-4})
`-holonomy-min-plaquettes`	Min. internal plaquettes to attempt correction (default: 2)
`-no-cache`	Disable in-memory wavefunction cache
`-np`	Number of processes to use (default: 1)
`-o file_path`	Name of output log file. Extension will be *.log* regardless of user input
`-flush`	Flush the output to stdout
`-v`	Increase output verbosity

In the end we get two files *dpc.npy* and *dp.npy*:

- *dpc.npy* contains for each pair of k-points and bands the dot product of the Bloch factors
- *dp.npy* contains the modulus of *dpc.npy*

The options for `dot` are shown in table 4.5

Table 4.5: Options for script `dot`

`-h, --help`	Show a help message and exit
`-np`	Number of processes to use (default: 1)
`-o file_path`	Name of output log file. Extension will be ignored.
`-flush`	Flush the ouput to stdout
`-v`	Increase output verbosity

6 Find the bands

To find which eigenvalues/eigenfunctions have continuity, one runs script `cluster`.

```
berry cluster [Mb] [script options]
```

This script uses graph theory and unsupervised machine learning to establish electronic bands that are analytic in k-space.

It uses data from files *dp.npy* and *eigenvalues.npy* to establish continuity between wavefunctions of neighboring k-points.

The optional positional parameter `Mb` is the maximum band to consider; if omitted, all bands up to the highest one are used. The minimum band is set with the option `-mb`.

It produces the following files:

bandsfinal.npy stores an array bandsfinal[k,b] = number of the original (DFT) band that is continuous to band b at k-point k.

signalfinal.npy signalfinal[k,b] is a measure of the quality of the attribution to a band, on the dot-product scale (see table D.1).

correct_signalfinal.npy the same quality measure on the energy-continuity scale (see table D.2).

degeneratefinal.npy the list of k-points that need a basis rotation.

final_score.npy the mean dot-product score of each band.

completed_bands.npy the bands that were clustered free of genuine errors (usable directly).

final.report a human-readable summary, with a per-band status table.

A detailed explanation of the algorithm is given in the appendix Algorithm of `cluster` script.

The options for `cluster` are shown in table 4.6.

Table 4.6: Options for script `cluster`

`-h, --help`	Show a help message and exit
`Mb`	Maximum band to consider (positional; default: highest band)
`-mb`	Minimum band to consider (default: 0)
`-np`	Number of processes to use (default: 1)
`-t [0.0-1.0]`	Tolerance used for graph construction (default: 0.80)
`-step [0.0-1.0]`	Step for the alpha sweep (default: 0.5, i.e. 1.0 → 0.5 → 0.0)
`-alpha [0.0-1.0]`	Initial alpha value (default: 1.0)
`-o file_path`	Name of output log file Extension will be .log
`-flush`	Flush the ouput to stdout
`-v`	Increase output verbosity

7 Basis rotation

Where two or more bands are degenerate, the individual wavefunctions are only defined up to an arbitrary unitary mixing within the degenerate subspace, and the DFT output

picks that mixing arbitrarily at each k-point. A basis rotation has to be applied to restore continuity across k-space.

For this, `basis` can be run, after `cluster`:

```
berry basis [Mb] [script options]
```

The script `basis` is the *a-posteriori* twin of `degenrot` (section Degenerate subspace basis rotation): it applies exactly the same subspace-rotation machinery, but instead of detecting degeneracies from the eigenvalues it takes its list of degenerate k-points from the clustering output *degeneratefinal.npy*. This lets it fix the specific points that the band classification flagged as needing a rotation, rather than every energy degeneracy in the mesh.

The algorithm proceeds in the same phases described in section Degenerate subspace basis rotation, and only the first (detection) phase differs:

Phase 1 — read the flagged points. Each row of *degeneratefinal.npy* is a degenerate band pair $[k, b_1, b_2]$. Overlapping pairs that share a band are merged into the maximal degenerate group (so a triple degeneracy stored as three pairs becomes one subspace of three bands). A flagged pair whose two bands are farther apart in energy than the grouping threshold `-gethr` (default 0.01 eV) is treated as non-degenerate and dropped, so a distant band is never pulled into the subspace.

Phase 2 — form zones. Adjacent flagged k-points sharing the same degenerate band group are joined into a connected *zone*; a flagged point with no flagged neighbour is an *isolated* single-point zone.

Phases 3–4 — reference and propagation. Within each zone the subspace is aligned to a clean (non-degenerate) reference neighbour with an SVD/Procrustes rotation, and the rotation is propagated across the zone by breadth-first search, with the holonomy correction and iterative gauge refinement described in section Degenerate subspace basis rotation.

Phase 5 — output. A per-zone report is printed and the zone map is saved to *basisrotation_zones.npy*.

Only the flagged k-points are ever modified; clean neighbours are read as reference anchors but never changed. The rotated wavefunctions **overwrite the *.wfc* files in place** (there is no *.wfc1* file). Both colinear and non-colinear calculations are supported: for the non-colinear case the same $N \times N$ unitary is applied to both spinor components.

The options for `basis` are shown in table 4.7

Table 4.7: Options for script `basis`

`-h, --help`	Show a help message and exit
`Mb`	Maximum band to consider (positional)
`-np`	Number of processes to use (default: 1)
`-gethr eV`	Energy-coherence threshold for grouping flagged bands into one subspace (default: 0.01; negative disables the guard)
`-c`	Compress the output files
`-o file_path`	Name of output log file Extension will be .log
`-flush`	Flush the ouput to stdout
`-v`	Increase output verbosity

8 Convert wavefunctions to k space

With the bands well defined, it is now necessary to proceed to calculate the wavefunctions in k-space and their gradients.

For that, run `r2k`:

```
berry r2k highest_band [script options]
```

It will calculate from band 0 to the value of the band given (`highest_band`), unless a minimum band is given. The highest band has to be given because in principle not all bands that were initialy calculated will be complete, and it doesn't make sense to use them in further calculations.

The wave functions originaly are saved for each k-point in a file, and the file contains the value of the wavefunction in each point of the real space. This program converts this format to one where for each point in real space there is a file with all the values of wavefunctions of the same band as a function of k-points, which is suitable to diferentiate.

$$\Psi_{\boldsymbol{k}}(\boldsymbol{r}) \rightarrow \Psi_{\boldsymbol{r}}(\boldsymbol{k})$$

Will save the wavefunctions of each band in a compressed file *wfcpos#.npy* where # = number of band and the gradients of each band in a compressed file *wfcgra#.npy* where # = number of band. The options for `r2k` are shown in table 4.8.

9 Calculation of the Berry connections and curvatures

The Berry connections and curvatures are calculated using the script `geometry`.

```
berry geometry highest_band [script options]
```

Table 4.8: Options for script `r2k`

`-h, --help`	Show a help message and exit
`-np`	Number of processes to use (default: 1)
`-mb`	Minimum band to consider (default: 0)
`-o file_path`	Name of output log file Extension will be ignored
`-flush`	Flush the ouput to stdout
`-v`	Increase output verbosity

The highest band is the number of the last band to be used in the calculations. In principle, it will be the same as the one used in `r2k`. The options for `geometry` are shown in table 4.9.

Table 4.9: Options for script `geometry`

`-h, --help`	Show a help message and exit
`-np`	Number of processes to use (default: 1)
`-mb`	Minimum band to consider (default: 0)
`-prop` [both,connection,curvature]	Specify property to calculate. (default: both)
`-o file_path`	Name of output log file Extension will be ignored
`-flush`	Flush the ouput to stdout
`-v`	Increase output verbosity

10 Optical conductivity and second harmonic generation

To calculate the linear condutivity, the script `condutivity` should be used and for the second harmonic condutivity, the script `shg`.

```
berry conductivity highest_band [script options]
```

and

```
berry shg highest_band [script options]
```

The highest band is the number of the last band to be used in the calculations, and has to be higher than the valence band. There are several options for the calculation that can be changed and are listed in table 4.10.

These scrips calculate the conductivity between 0 and the maximum energy (in Ry), with a step (energy step) and a broadning.

The details of these calculations can be found in

`https://arxiv.org/abs/2006.02744`

Table 4.10: Options for scripts `conductivity` and `shg`.

`-h`, `--help`	Show a help message and exit
`-np`	Number of processes to use (default: 1)
`-eM`	Maximum energy in Ry units (default: 2.5)
`-eS`	Energy step in Ry units (default: 0.001)
`-brd`	Energy broading in Ry units (default: 0.01)
`-o file_path`	Name of output log file Extension will be ignored
`-flush`	Flush the ouput to stdout
`-v`	Increase output verbosity

Appendices

Installing for noncolinear calculations

For noncolinear calculations, the fortran script `wfck2r.f90` has to be edited, because currently it only prints the first function of the spinor, and for **berry** calculations we need the full spinor in real space.

The steps for editing and compiling the fortran scipt are the following:

First, create a copy of `wfck2r.f90` located in PP/src of the QUANTUM ESPRESSO's home directory and name it `wfck2rFR.f90` in the same directory.

In the wfck2rFR.f90 file, on line 252, between `enddo` and `endif`, add in new lines:

```
if (noncolin) then
  do i3 = 1, dffts%nr3x, nevery(3)
  do i2 = 1, dffts%nr2x, nevery(2)
  do i1 = 1, dffts%nr1x, nevery(1)
    !val = local_average()
    val=dist_evc_r(i1+(i2-1)*dffts%nr1x+(i3-1)*dffts%nr1x*dffts%nr2x,2)
    write(iuwfcr+1,'("(",E20.12,",",E20.12,")")') val
  enddo
  enddo
  enddo
endif
```

So this is immediatly after writing the wavefunction (first of the spinor), and simply replicates that writing but for the second wavefunction of the spinor.

The first wavefunction of the spinor is on array `dist_evc_r(:,1)` and the second is on `dist_evc_r(:,2)`.

Now the file needs to be compiled using the same configuration as the rest of QUANTUM ESPRESSO. For this, one needs to edit the `Makefile` that is in the same directory PP/src of the QUANTUM ESPRESSO's home directory.

First we add `wfck2rFR.x` to the list of executables listed under the tag `all :`

Then add the following lines (for instance, bellow the equivalent lines for `wfck2r.x`):

```
wfck2rFR.x : wfck2rFR.o libpp.a $(MODULES)
```

```
$(LD) $(LDFLAGS) -o $@ \
        wfck2rFR.o libpp.a $(MODULES)  $(QELIBS)
- ( cd ../../bin ; ln -fs ../PP/src/$@ . )
```

(beware that tabs must be used on second to fourth line, and not spaces.)

After this, to compile, just execute the command on the directory PP/src:

```
make wfck2rFR.x
```

and it's done. You shuld see a link to the executable `wfck2rFR` on the `bin` directory of Quantum ESPRESSO.

Glossary

This glossary indicates the meaning of the variables used in the suite **berry**.

npr	Number of processors for the run
nkx	Number of k-points in the x direction
nky	Number of k-points in the y direction
nkz	Number of k-points in the z direction
nks	Total number of k-points
nr1	Number of points of wfc in real space x direction
nr2	Number of points of wfc in real space y direction
nr3	Number of points of wfc in real space z direction
nr	Total number of points of wfc in real space
k0	Initial k-point (coordinates) (2pi/bohr)
step	Step between k-points (2pi/bohr)

dftdirectory	Directory of DFT files
name_scf	Name of scf file (without suffix)
name_nscf	Name of nscf file (without suffix)
wfcdirectory	Directory for the wfc files
prefix	Prefix of the DFT QE calculations
outdir	Directory for DFT saved files
dftdatafile	Path to DFT file with data of the run
berrypath	Path of BERRY files

a1	First lattice vector in real space (bohr)
a2	Second lattice vector in real space (bohr)
a3	Third lattice vector in real space (bohr)
b1	First lattice vector in reciprocal space (2π/bohr)
b2	Second lattice vector in reciprocal space (2π/bohr)
b3	Third lattice vector in reciprocal space (2π/bohr)

nbnd	Number of bands
eigenvalues	Array with all eigenvalues of the calculation (Ry)
occupations	Array with all occupations of the calculation
phase[i,nk]	Array with exp(1j*(kpoints[nk,0]*r[i,0] + kpoints[nk,1]*r[i,1] + kpoints[nk,2]*r[i,2]))
kpoints[nk,2]	Array with coordinates of all k-points (2π/bohr)
r[i,2]	Array with coordinates of all points of space (bohr)
neig[nk,3]	Array with number of the four k-points around nk

List of files in the suite

All programs of this package are in python 3.

The package contains the python scripts in the main berry directory:

- preprocessing.py
- generatewfc.py
- degenrotation.py
- dotproduct.py
- clustering_bands.py
- basisrotation.py
- r2k.py
- berry_geometry.py
- condutivity.py
- shg.py

and the auxiliary files:

- cli.py
- __init__.py
- _version.py

Visualization scripts are in directory 'vis':

- _debug.py
- _geometry.py
- _wave.py

Also the subroutines (depend on the other scripts) are in directory '_subroutines':

- headerfooter.py
- contatempo.py
- parserQE.py
- loaddata.py
- loadmeta.py
- comutator.py
- write_k_points.py
- clustering_libs.py
- clustering_signaling.py

Some utilities are in directory 'utils':

- emailSender.py
- jit.py
- _logger.py

It is likely that some of these scripts will be merged with others while new ones will appear due to new functionalities.

Algorithm of `cluster` script

1 Algorithm

This solution employs different techniques to solve the bands crossing problem as graph theory and unsupervised machine learning methods. The bands clustering algorithm uses the information of k-points' wavefunctions, the dot products of their Bloch factors, and their corresponding band energy. Thus, the solver algorithm iterates between two main pieces: the component identification based on graph theory and the samples assignment using unsupervised machine learning methods.

The solver's objective is to get closer to the best solution in each iteration by maximizing the number of solved bands.

The bands clustering algorithm uses the wave functions, their dot products, and their corresponding band energy for these calculations.

The output is a set of files that will be used by other programs (*bandsfinal.npy*, *signalfinal.npy*, *correct_signalfinal.npy*, *degeneratefinal.npy*, *final_score.npy* and *completed_bands.npy*), plus a human-readable *final.report*; they are described below.

In the following sections we will discuss in more detail the purpose of each band clustering algorithm's part and how it works.

1.1 Problem configuration and notation definition

We first need to organize the information problem in a convenient data structure. The essential elements we have are the energies $E_{k,n}$ for each k-point $\mathbf{k} \equiv (k_x, k_y)$ and band n, the k-points' neighbor identification, and the dot product between the neighboring Bloch factors. The *loaddata* module gives the algorithm most of this information; however, this module does not contain the dot product data; it is in the *dp.npy* file.

First, we create a mixed entity using a k-point and its band energy as if it were a vector on a "bands-energy" space. In the following, a variable $\mathbf{p}$ identifies a point in this space. Thus, this variable $\mathbf{p}$ contains the k-point coordinates and an eigenenergy of the k-point (which is a band designation).

$$\mathbf{p} \equiv (k_x, k_y, E_{k,n}) \tag{D.1}$$

The set of all k-points identifiers (k indices) is K, where $K \equiv [k = 0, \ldots, N-1]$, and $\mathbf{K} \equiv \{\mathbf{k}_0, \mathbf{k}_1, \ldots, \mathbf{k}_{N-1}\}$ is the set of all k-points vectors. N is the total number of k-points. It is possible to define an index p which uniquely determines a wavefunction' Bloch factors:

$$p = n\,N + k \tag{D.2}$$

We call P to the set of all p's. n is the original band as attributed by the DFT software; we define the set of n's: $B \equiv [0, n_B - 1]$. n_B is the total number of bands.

Then

$$P \equiv [0, n_B N - 1] \tag{D.3}$$

It leads to

$$E_{k,n} = E_p, \qquad k \in K,\ n \in B,\ p \in P \tag{D.4}$$

Let us consider the points p and $q \in P$, with q is neighbor of p (by neighbor we mean that the k-points that are implicit in the varaibles are adjacent in x or y directions in reciprocal space); the dot product between their Bloch factors is given by $0 \leq |\langle p|q\rangle| < 1$. If the Bloch factors p and q belong to the same band, and the band is analytic, since they are close together, their dot product should be close to one. On the other hand, this dot product should be close to zero if p and q belong to different bands, since different bands are orthogonal.

Thus, this data seems to give all the information necessary to solve the band-crossing problem. However, from a computational point of view, more is needed; during the implementation of this solution, several problems were found, and just a few of them were solved successfully.

Regardless, the dot product information is not wholly needless for values close to one $|\langle p|q\rangle| \approx 1$; it is possible to form groups of well-constructed groups of points. The existence of points and a way of measuring relations between them reminds the graph concept. This is the motivation for one of the algorithm's parts: use graph theory methods to identify these groups and, after that, use unsupervised machine learning techniques to classify them.

Consequently, this information makes it possible to construct the first data structure. It is a graph $G(V, E)$ (V is a set of vertices or nodes, and E is a set of edges (connections)) where each point p is a node, so that $V = P$, and if the connection (dot-product) is more significant than an adequate tolerance TOL, there is an edge between these nodes. For simplicity, this graph is undirected and does not have weights on its edges.

$$E = \{(p, q) \mid p, q \in P \ \ \wedge \ \ |\langle p|q\rangle| > \mathrm{TOL}\} \tag{D.5}$$

Therefore, the initial state of the problem is prepared in a convenient data structures. The next step is the solver algorithm that will use that data and the graph previously

constructed as the initial solution point. The solver algorithm will iterate over this information.

1.2 Solver Algorithm

The solver algorithm (SA) is an optimization process based on iteration for solving the band-crossing problem. As discussed before, this algorithm iterates between two parts. One piece is based on graph theory (GT), and the other is based on unsupervised machine-learning (UML) techniques. The initial conditions for this algorithm were described in the previous section. This SA follows the flux chart in figure D.1.

The SA can be simply described as various iterations starting in an initial state, and as a result some points are not attributed to their initial band and others are noted as having a strong connection between them. We call components to the sets of points with connections and if there are no connections, the single point is also considered a component. In the first iteration, these components were computed by constructing the graph using the dot-product information only.

The initial state of an iteration is the best result of the previous iterations, since SA is an optimization algorithm.

One SA iteration involves identifying the components, given the initial state in the form of a graph. Then, clustering these components gives larger components, that will form a new state that will be used as a initial state of the next iteration. The optimization process ends when the result does not change between iterations, reaching a stationary solution or the relaxation parameter reaches the minimum value. This relaxation value is a parameter for the second part of the algorithm, and will be discussed in more detail later.

1.3 Identification of problems and component detection

The concept of a component in GT is necessary to understand this part of the SA. A component is a sub-graph (i.e., a smaller graph g belonging to a bigger one G where $g \subset G$), with edges (connections) between its nodes, but no connections with the nodes of other components, as shown in the example in figure D.2.

These components are straightforward to identify using existing GT techniques, but as expected, they are not the final result are just portions of a band. For this reason, it is essential to classify these components to form completed bands. Hence, the SA uses UML techniques to perform the assignment.

This algorithm gives very good results for some materials; however, there are problems. Namely, a few points create "loops" between prohibited points. It is a prohibited path because, there should be no connection between two points in the same k-point (that is two wavefunctions with the same k-point must not belong to the same band). This results in the algorithm assuming two components that should be separated as a single

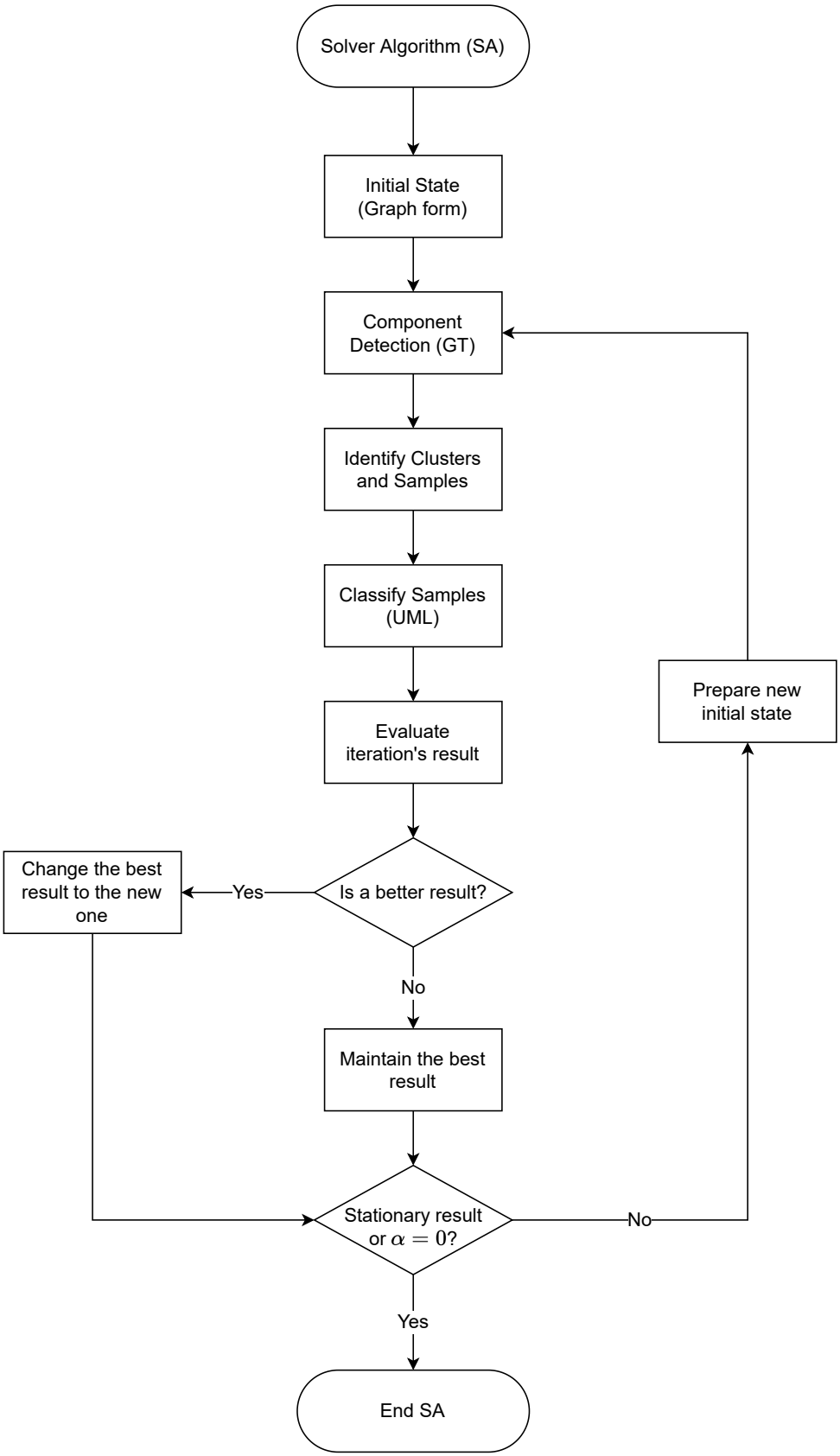
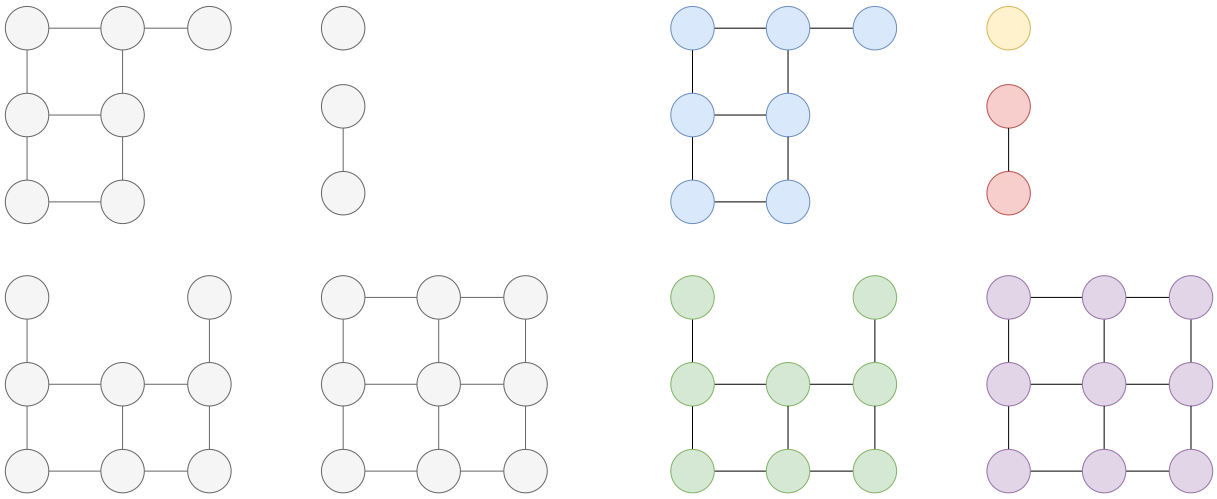


Figure D.1: Flowchart with a simplified description of the solver algorithm.



(a) (b)

Figure D.2: Figure

component.

Two types of points create these "loops." One of them is straightforward to identify; they are degenerate points up to a numerical error (i.e., for p and $q \in P$ and the same k-point, their energy is numerically the same, $E_p = E_q$). Thus, it is only necessary to identify them and see if they create these forbidden paths. When found, the connection is broken.

The second type of points are not strictly degenerate but have the same behavior, complicating their detection. These more peculiar points are in high energy bands and, besides other inconveniences, limit the algorithm to obtain better results. Nonetheless, for both types of problems, we need a more substantial justification for this behavior.

1.4 Unsupervised Machine Learning Techniques to Band Clustering

Since the SA uses UML techniques to classify the components, it is necessary to establish a clear notation. We call clusters the components with many points that can not join to another large component, because they are geometricaly incompatibles. These clusters are electronic bands of eingenenergies with part of their points attributed. We call samples to the rest of components, that can be assigned to clusters. When a cluster has the number of nodes (points p) equal to the total number of k-points, N, this cluster is solved (the band is solved). Otherwise, it is an incomplete cluster, and the sample assignment tries to complete it.

Supervised machine learning methods require large quantities of labeled examples, which we do not have. Thus, this problem lies in the unsupervised machine learning (UML) problems category. Consequently, this SA's part aims to identify the samples that complete the clusters using these UML methods.

The UML method needs a way to compare samples to clusters and evaluates the at-

tribution. Therefore, let us define the similarity between two objects, X and Y, $\mathcal{S}(\mathrm{X},\mathrm{Y})$. The $\mathcal{S}(\mathrm{X},\mathrm{Y})$ value measures how similar these objects are based on one chosen metric. When the similarity is higher, the objects are more similar. We will use a normalized similarity. So, the $\mathcal{S}$ value must satisfy the following properties:

1. It is positive, $\mathcal{S}(\mathrm{X},\mathrm{Y}) > 0$
2. It is symetric, $\mathcal{S}(\mathrm{X},\mathrm{Y}) = \mathcal{S}(\mathrm{Y},\mathrm{X})$
3. It is maximum when $\mathrm{X} = \mathrm{Y}$, $\mathcal{S}(\mathrm{X},\mathrm{X}) = 1$

On the other hand, we use a similarity metric between samples and clusters that only needs their representative points. A cluster has many well-connected points, but only a few are required for comparison with a candidate sample to join the cluster. Only the cluster and sample boundary points may establish relations between them. Then, these are the representative points.

Identifying the representative points is an undersampling technique. We use the convolution of the Sobel operator, (G_x, G_y), with the cluster's k-space projection A to determine their edges. The matrix A has the shape of k-space; each matrix's entry, A_{k_x,k_y}, is one if the cluster includes a point $\mathbf{p}(p)$ with that k-point and zero otherwise.

This operator is a simple discrete gradient that identifies high-frequency variations. These variations define the cluster's edges.

$$A_{k_x,k_y} = \begin{cases} 1, & \mathbf{k} \in \mathbf{K}_{\text{Cluster}} \subset \mathbf{K} \\ 0, & \text{otherwise} \end{cases} \tag{D.6}$$

The Sobel operator G_x and G_y is defined as:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \tag{D.7}$$

$$G_y = G_x^T = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{D.8}$$

Then, the gradient on the $\alpha = \{x, y\}$ direction of A is

$$A_\alpha = G_\alpha * A, \qquad \alpha = \{x, y\} \tag{D.9}$$

where $*$ is the convolution operator and the representative cluster edges are given by.

$$\text{edges} = \sqrt{A_x^2 + A_y^2} > 0 \tag{D.10}$$

Now, we can explicitly choose a metric to define the similarity that we will use. A similarity score between neighboring representative points is a weighted mean between

the modulus of the dot product $|\langle p|q\rangle|$, and a function $f_p(E_q)$ of the energies E_q of the neighbor point q.

The similarity of a cluster with a sample is defined as the mean of the individual similarity score between neighboring representative points.

Thus, the similarity score, s_p^q, between the cluster's $p \in P$ point and its sample's $q \in P$ neighbor is:

$$s_p^q = \alpha\,|\langle p|q\rangle| + (1-\alpha)\,f_p(E_q) \tag{D.11}$$

The value of α is a relaxation parameter in the optimization process. Its initial value is 1 (so the first iterations rely on the dot product alone), and it is swept down to its minimum value of 0 in steps of size `step` (default 0.5, giving the sweep $1 \to 0.5 \to 0$). The sweep is convergence-paced: α is held at its current value while the solution keeps improving and is only lowered after it stalls for a few consecutive iterations, so lowering α gradually shifts trust from the dot product toward energy continuity.

The function $f_p(E_q)$ scores the energy continuity between the energy E_{best} that the cluster's trajectory predicts at q's k-point and the candidate energy E_q. It is a smooth, scale-aware match with a gross-jump veto:

$$f_p(E_q) = \begin{cases} 0, & \Delta E^q_{\text{best}} > E_{\text{accept}} \\ \dfrac{\sigma}{\sigma + \Delta E^q_{\text{best}}}, & \text{otherwise} \end{cases} \qquad \Delta E^b_a = |E_a - E_b| \tag{D.12}$$

where σ is the local one-step energy dispersion (the 99.9th percentile of $|E(k+1) - E(k)|$) and $E_{\text{accept}} \approx 3\sigma$ is a loose gross-jump guard: a candidate on the far side of an energy wall (a genuine inter-band gap) scores 0 and is never joined. This replaces an earlier objective, $\min\{\Delta E^i_{\text{best}}\}_i / \Delta E^q_{\text{best}}$, which scored how close E_q was to being the *globally nearest* band; that measure was non-directional and became unreliable when the gap collapses below σ (spin–orbit / Kramers manifolds), where the nearest band is no longer the continuous one.

We consider a smooth variation of bands in bands-energy space. Thus, the energy profile in each band's **p** point can be approximated by a second-order polynomial. This approximation allows us to calculate the best energy value E_{best} by extrapolation from the neighboring points that belong to the cluster. If there are not enough points to fit the curve, the nearest neighbor's energy is used. The gross-jump guard E_{accept} is deliberately not keyed to the band gap: gating on the gap once shattered the graph, so energy is kept out of the graph edges and used only through $f_p(E_q)$ and the a-posteriori repair described below.

Then, the similarity $\mathcal{S}(\mathrm{C}, \mathrm{S})$ between the cluster C and sample S is

$$\mathcal{S}(\mathrm{C},\mathrm{S}) = \frac{1}{4\tilde{N}} \sum_{p\in\mathrm{C}} \sum_{q\in\mathrm{S}} s_p^q, \qquad \mathrm{C},\mathrm{S} \subset P \tag{D.13}$$

Where $\tilde{N}$ is the total number of points in the sample's edge. The factor of 4 appears because each q point generally contains four neighbors.

Assigning samples to clusters uses similarity values, trying to maximize it between them. Hence, all the samples are compared to all clusters to obtain their similarities. The classification is made iteratively. For each iteration, one sample is assigned to the best cluster. After that, the similarity scores are recalculated between the samples and the modified cluster.

This UML algorithm is the SA's bottleneck in executing time. This complication comes from the fact that this part is mostly computed linearly (i.e., the result on each iteration depends on the previous result). Therefore parallelizing it is challenging.

Ultimately, all the samples are assigned to one cluster leading to a band solution. However, as mentioned before, many problematic points exist. In particular, high energy bands usually have many oscilations, and in this case the assumption of smooth variation is inaccurate, and the metric criteria has to be improved.

1.5 Evaluation and optimization of the result

The full SA is determinist because the result is always the same for the same conditions. However, after one SA iteration (GT and UML methods), it obtains one solution for the band-crossing problem. Different iterations obtain different number of bands completed and correct, which we want to maximize. Bands may have points wrongly attributed to them, and so an verification and evaluation is needed.

We use two stages to evaluate the iteration result. The first one considers only the dot-product information. It identifies and signals points acording to their assignment as indicated in table D.1. A point is correctly assigned when the mean dot-product $\overline{|\langle p|q\rangle|}$ is close to one.

Table D.1: Signals for the attribution of points to bands.

Signal	Description	Condition
0	Not solved	
1	Wrong	$\overline{\lvert\langle p\rvert q\rangle\rvert} \leq 0.2$
2	Degenerate	
3	Potential mistake	$0.2 < \overline{\lvert\langle p\rvert q\rangle\rvert} \leq 0.8$
4	Potential correct	$0.8 < \overline{\lvert\langle p\rvert q\rangle\rvert} \leq 0.9$
5	Correct	$\overline{\lvert\langle p\rvert q\rangle\rvert} > 0.9$
6	Forced	force-filled by the completeness pass

This stage signals some points that require more analysis, namelly the ones that are signaled as potentialy correct or mistaken. The second stage of evaluation analyzes these points to decide if they are effectively correct or not. This evaluation uses the energy continuity criteria (i.e., the bands need to be analytical in bands-energy space). Using the energy profile approximation by a second-order polynomial, it is possible to evaluate the continuity for each point's direction. Thus, each point receives a new signal value related to the number of directions D, that preserves the energy continuity.

Table D.2: Signals for the attribution of points to bands.

Signal	Description	Condition
0	Not solved	
1	Wrong	$D = 0$
2	Degenerate	
3	Other	$0 < D < 4$
4	Correct	$D = 4$
5	Forced	force-filled by the completeness pass

Furthermore, the total solution (all solved bands) is scored using the dot-product criteria. The score is the mean overall $\overline{|\langle p|q\rangle|}$ of the p-points of the band. This new information makes it possible to compare the final solution with the one of the previous iteration.

We consider the best result the one that reaches more solved bands and a larger score for each band. Consequently, this result will be the starting point for the next iteration.

The points not attributed, the mistakes, and the doubtful points became nodes with no connections. The points signaled as correct form well-constructed bands' portions. Thus, these points are nodes with edges between them. Accordingly, the collection of these nodes and edges forms the new graph. Therefore, the new initial state is ready and compatible with the first step of the SA (it needs the data in a graph form).

The next iterations will try to solve the wrong attributions to obtain a better result until it reaches a stationary point or the relaxation criteria ends.

1.6 A-posteriori repair and final classification

When the iterative solver stops, its best result may still contain points that failed the energy-continuity test or that were left unattributed. Three passes run once, on that best result, and they never touch a point that already validated cleanly:

1. **Energy-continuity repair.** Only the flagged (wrong / not-solved) points, and points holding a valid but energy-discontinuous band, are freed and jointly reassigned. Each empty slot's expected energy is extrapolated by a quadratic fit built from *trusted* neighbors only, and the freed bands are redistributed to the slots they

best match, never reusing a band twice at the same k-point, and only when the reassignment stays within the gross-jump guard E_{accept}. Repaired points become trusted, so the fix propagates inward, one layer per round.

2. **Degenerate-group completion and bijection.** Inside a near-degenerate group of bands (energies within a few meV) the individual band label is a gauge choice, so the group's slots are completed by a within-group bijection. The full attribution is then made a per-k permutation (each original band used in at most one slot): duplicated bands keep their best copy and the losers move to a missing band by best energy fit.

3. **Force completion.** Any slot still empty is force-filled with the closest available band in energy, using the continuity extrapolation as a reference. These points are marked *Forced* (signal 6 in table D.1, 5 in table D.2); they are not genuine solves and are flagged as such.

Finally each band is graded in the *final.report* into one of: `CLEAN` (pristine), `USABLE` (no genuine errors; may carry a sub-pristine score or benign force-fills inside a degenerate doublet), `ROTATE` (usable after a basis rotation), `CHECK` (a force-fill across a real energy gap — verify), or `FAIL` (a genuine energy break or a low score). The `CLEAN`/`USABLE`/`ROTATE` bands are the ones saved in *completed_bands.npy*.

2 Bands Clustering program (BCP)

The python program *clustering_bands0.py* implements the steps needed to use the algorithm explained previously. This program reads and prepares all the dependencies to perform the bands clustering algorithm, which is implemented in the *clustering_libs.py* module.

The BCP uses the results of the previous berry suit algorithms. The needed results from the previously ran programs are:

Table D.3: Previous berry suit programs save their results in various files. This table shows the required ones.

Description	File
The material's main data	*datafile.npy*
The modulus of Bloch's factor dot products $\lvert\langle p\rvert q\rangle\rvert$	*dp.npy*
The eigenvalues	*eigenvalues.npy*
A list with the neighbors of each k-point	*neighbors.npy*

The berry suite must be installed in the python environment. Then, the following command line executes the BC program in the material directory.

```
$ berry cluster
```

Note that running the algorithm is only possible if all the dependencies exist.

Also, it is possible to change some of the default behaviour by passing some parameters, which are shown in table D.4.

Table D.4: Parameters that are received as optional arguments to the BCP.

Description	Flag	Default
The maximum band to solve	Mb	highest band
The minimum band to consider	-mb	0
Number of processes to use	-np	1
Dot-product tolerance used for graph construction	-t	0.80
Step for the alpha sweep	-step	0.5
Initial alpha value	-alpha	1.0

Bibliography

[1] M. Gradhand, D. V. Fedorov, F. Pientka, P. Zahn, I. Mertig, B. L. Györffy, First-principle calculations of the berry curvature of bloch states for charge and spin transport of electrons, Journal of Physics: Condensed Matter 24 (21) (2012) 213202. doi:10.1088/0953-8984/24/21/213202.

[2] C. Aversa, J. E. Sipe, Nonlinear optical susceptibilities of semiconductors: Results with a length-gauge analysis, Physical Review B 52 (1995) 14636–14645. doi:10.1103/PhysRevB.52.14636.

[3] J. E. Sipe, A. I. Shkrebtii, Second-order optical response in semiconductors, Physical Review B 61 (2000) 5337–5352. doi:10.1103/PhysRevB.61.5337.

[4] G. B. Ventura, D. J. Passos, J. M. B. Lopes dos Santos, J. M. Viana Parente Lopes, N. M. R. Peres, Gauge covariances and nonlinear optical responses, Physical Review B 96 (2017) 035431. doi:10.1103/PhysRevB.96.035431.

[5] P. Giannozzi, S. Baroni, N. Bonini, M. Calandra, R. Car, C. Cavazzoni, D. Ceresoli, G. L. Chiarotti, M. Cococcioni, I. Dabo, et al., Quantum espresso: a modular and open-source software project for quantum simulations of materials, Journal of Physics: Condensed Matter 21 (39) (2009) 395502. doi:10.1088/0953-8984/21/39/395502.

[6] P. Giannozzi, O. Andreussi, T. Brumme, O. Bunau, M. Buongiorno Nardelli, M. Calandra, R. Car, C. Cavazzoni, D. Ceresoli, M. Cococcioni, et al., Advanced capabilities for materials modelling with quantum espresso, Journal of Physics: Condensed Matter 29 (46) (2017) 465901. doi:10.1088/1361-648X/aa8f79.

[7] P. H. Schönemann, A generalized solution of the orthogonal Procrustes problem, Psychometrika 31 (1) (1966) 1–10. doi:10.1007/BF02289451.

[8] Å. Björck, G. H. Golub, Numerical methods for computing angles between linear subspaces, Mathematics of Computation 27 (123) (1973) 579–594. doi:10.1090/S0025-5718-1973-0348991-3.

[9] G. H. Golub, C. F. Van Loan, Matrix Computations, 4th ed., Johns Hopkins University Press, Baltimore, 2013. ISBN 978-1-4214-0794-4.

[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, Cambridge MA, 2009. ISBN 978-0-262-03384-8.

[11] H. F. Walker, P. Ni, Anderson acceleration for fixed-point iterations, SIAM Journal on Numerical Analysis 49 (4) (2011) 1715–1735. doi:10.1137/10078356X.